

DreamOS: A Locally Run, Privacy-Verifiable Semantic File Management System with Knowledge Graph and Scale Evaluation

Zaid Ali Syed, Om Vyas, Krushna Kadam

*Department of Information Technology, Institute of Information and Communication Technology (IICT)
MGM University, Chhatrapati Sambhajinagar, India
sidzaid72@gmail.com*

Abstract: Desktop file systems organize documents by folder and filename chosen at the moment of saving, and these names rarely describe what a file is about months later. We present DreamOS, a locally run desktop file manager that retrieves, opens, relates and organizes files by natural-language meaning rather than by name. DreamOS embeds file content with a local sentence-embedding model, routes each request through an intent classifier, and exposes six modules: a natural-language interface, a semantic search engine, an AI file organizer, a knowledge graph of file relationships, a context memory engine, and an intelligent workspace manager. Unlike prior semantic file-system work, which targets software agents through an API, DreamOS exposes the same capability through one conversational interface for a human user, and reports a live, checkable audit of whether any data could leave the machine. We evaluate the system on synthetic collections of 500, 2,000 and 5,000 files using the real local models, and report two defects exposed by that evaluation and their fixes: a name-resolution delay that added roughly one second to every model call, and a pairwise knowledge-graph computation that scaled quadratically with the number of files. After vectorizing the graph computation, rebuild time at 2,000 files fell from 246.1 s to 24.8 s; at 5,000 files it is 143.3 s, which we report as the system's current scaling limit rather than a solved problem. A third defect, in which a language model's own judgement about conversational reference overrode a confident direct answer, was found during a live rehearsal and fixed by giving deterministic evidence precedence over the model's judgement.

Index Terms: semantic search, sentence embeddings, knowledge graph, local large language models, personal information management, privacy by design, information retrieval.

I. INTRODUCTION

As the number of files on a personal computer grows into the thousands, filenames stop matching how a person remembers the content. A user who saved `resume_final_v3.txt` six months ago must now recall that exact name to find it, rather than describing what the file contains. Mainstream desktop search tools remain keyword- and metadata-driven: a query must share vocabulary with the target file [5], [6], [7]. Research on semantic file systems has shown that representing files as embedding vectors and retrieving them by similarity is workable [1], but this prior work is designed for software agents consuming an API, not for a person typing a request, and it is not demonstrated to run entirely on local models without a cloud dependency.

DreamOS addresses this gap with a desktop application that lets a user describe a file in plain language and get it found, opened, related to other files, or organized, with every

inference step running on the user's own machine. This paper makes the following contributions:

- 1) An architecture that exposes semantic file retrieval and action through a single conversational interface for a human user, rather than an agent API, with six cooperating modules.
- 2) A privacy audit that reports, from live configuration rather than from a fixed claim, whether a request could leave the machine, together with an independent network-level check.
- 3) A deterministic-precedence rule for resolving conversational follow-up questions, introduced after a live failure in which a language model's own judgement about reference overrode a confident direct search result.
- 4) A vectorized, equivalence-tested knowledge-graph construction method, and an empirical scale evaluation at up to 5,000 files that exposes and fixes two real performance defects.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the system architecture. Section IV describes the implementation of the six modules. Section V reports the scale evaluation and its findings. Section VI discusses limitations and future work, and Section VII concludes.

II. RELATED WORK

Dense text embeddings place semantically related text close together in a vector space. Mikolov et al. showed this for words [2]; Reimers and Gurevych extended the idea to sentences and paragraphs with Sentence-BERT, making whole-document comparison practical [3]. DreamOS uses a local embedding model in this same family (nomic-embed-text, served through Ollama) with the Chroma vector database for nearest-neighbour retrieval [4].

The closest published system is Shi et al.'s LLM-based Semantic File System (LSFS), presented at ICLR 2025 [1], which exposes embedding-vector file indexing as a set of APIs for an LLM agent to create, retrieve, update and roll up files using natural-language prompts. LSFS demonstrates that embedding indexes with an LLM front end can substitute for command-based file operations at the operating-system-agent level, which validates the architectural direction DreamOS also takes. LSFS targets agents consuming an API, however, not a human through a conversational surface, and its evaluation does not establish that the full pipeline runs on local models with no cloud dependency.

Consumer desktop search remains keyword- and metadata-driven. Windows Search indexes filenames, metadata and text content but matches lexically [5]; macOS Spotlight layers metadata and heuristic ranking over a similar keyword index [6]; Everything indexes the NTFS Master File Table for near-instant filename search but has no notion of file content [7]. None of the three retrieves by meaning, and none reports a checkable statement of what leaves the user's machine.

Taken together, the literature leaves three gaps that this paper addresses: embedding-based retrieval is demonstrated for agents rather than for a single conversational interface aimed at a person; privacy is typically asserted rather than reported from live, checkable configuration; and the behaviour of such systems on short queries and on collections of thousands of files is rarely measured, together with the defects that measurement exposes.

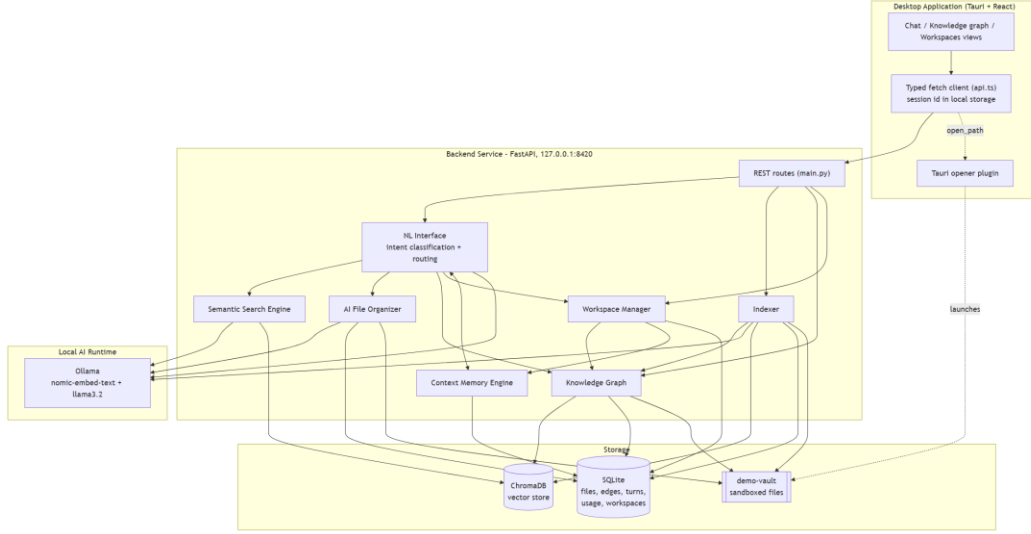


Fig. 1. DreamOS system architecture: desktop client, local service with six modules, local model runtime, and local storage.

III. SYSTEM ARCHITECTURE

DreamOS follows a client-server architecture confined to a single machine. A Tauri and React desktop client calls a local FastAPI service over HTTP on 127.0.0.1:8420, which in turn calls a local Ollama model runtime and two local data stores: SQLite for relational metadata, relationships, conversation and usage history, and ChromaDB for the embedding vectors of indexed text chunks (Fig. 1). No component in this path is reachable from outside the machine by default.

Six modules cooperate through the service. The Natural Language Interface (NLI) classifies each request into one of five intents (search, open, related, workspace, organize) using the local language model, and routes it. The Semantic Search Engine ranks files near the best embedding match for a query. The AI File Organizer proposes a category, tags and a summary for a file and applies the move only on explicit confirmation, with every move reversible. The Knowledge Graph computes typed relationships between files. The Context Memory Engine persists conversation turns and usage events so that follow-up questions resolve correctly. The Intelligent Workspace Manager turns graph clusters and usage patterns into named, explainable groups of files.

IV. IMPLEMENTATION

A. Semantic Search

A query is embedded and compared against every indexed chunk. Rather than a single fixed similarity

threshold, which we found drops correct answers to short queries, the search returns files within a margin of the best match above a floor, with a small score boost proportional to the fraction of query words that appear literally in a file's text or name. A result below a confidence threshold is labelled as low-confidence, and the open action never launches a file from a low-confidence result.

B. Knowledge Graph

Three typed, weighted edge kinds are computed between files: a similar edge from the cosine similarity of mean chunk embeddings after subtracting the corpus mean vector (mean-centering), a directed references edge when one file's text names another file by filename, and a shared_tag edge when two files share at least two AI-generated tags. Clusters are formed by average-linkage merging, which joins two groups only while the average edge weight between their members exceeds a threshold; this prevents a single weak bridge edge from joining two otherwise unrelated groups, a failure mode we observed with plain connected-component grouping.

The similarity computation was rewritten from a pairwise Python loop, which is quadratic in the number of files, to a single matrix product using NumPy: each file's vector is normalized, the full pairwise cosine similarity matrix is computed in one operation, and each file's strongest neighbours are selected with a partial sort. We verified this produces identical results to the original loop with a dedicated test over random vectors, so the rewrite changes only performance, not behaviour.

C. Context Memory and Deterministic Precedence

Every conversation turn is persisted with the identifiers of the files it showed, so a follow-up such as “open the second one” or “open it” can be resolved after a restart or after a file has been moved. Ordinal and bare-pronoun references are resolved by deterministic rule. A language model’s own flag for whether a message continues the prior conversation is consulted only as a last resort, after a confident direct search on the message’s own extracted content has already failed.

This ordering was introduced after a defect found during a live rehearsal: the message “what is related to my resume”, sent after an unrelated search for invoices, was answered about an invoice file. The language model had set its continuation flag to true for this message even though it correctly extracted “resume” as the topic, apparently because the sentence pattern matched an example given in its own

instructions. Because the system previously trusted that flag unconditionally, the stale context silently overrode a topic the query itself answered confidently. A regression test, which fails against the pre-fix code and passes after the fix, now guards this behaviour.

D. Privacy Audit

A privacy endpoint reads the live configuration and reports every network endpoint the service is configured to call, whether each is on loopback (127.0.0.1, ::1 or localhost), and what is stored locally. The audit states explicitly that it checks configuration rather than network traffic, and documents an independent check: inspecting listening sockets (for example with netstat) to confirm the service accepts connections only on loopback. This design lets the privacy claim be falsified rather than merely asserted.

used a fresh database and the real local models: nomic-embed-text for embeddings and llama3.2 for intent classification. We measured indexing throughput, knowledge-graph rebuild time, the latency of 36 search queries per run, and precision at 5 against the synthetic topic label. The benchmark script is part of the project repository so the measurements are reproducible. The service is additionally covered by 116 automated tests that run offline, with model calls replaced by fixed fixtures, and that run on every code push in continuous integration.

V. EVALUATION

A. Methodology

We generated synthetic vaults of 500, 2,000 and 5,000 text files across twelve topics (invoices, resumes, meeting notes, recipes, travel, fitness, source code, research notes, tax and finance, medical notes, school assignments, and shopping), with approximately half of the documents carrying one additional sentence drawn from an unrelated topic so that the topics are not trivially separable. Each run

TABLE I
Scale Benchmark After the Two Fixes Described in Section V-B

Files	Index (s)	Index (files/s)	Graph (s)	Search p50 (ms)	Precision@5
500	25.7	19.4	1.7	50.7	1.00
2,000	129.0	15.5	24.8	121.0	1.00
5,000	446.5	11.2	143.3	203.5	1.00

B. Two Defects Found by Measurement

The first run of this benchmark showed every embedding call taking approximately one second. Profiling located the time inside the socket connect call rather than in model inference or in the local HTTP round trip itself, which a direct call to the embedding endpoint confirmed completes in under 100 ms. The cause was that the configured hostname, localhost, resolved to the IPv6 loopback address first on the test machine, which the embedding service does not listen on, producing a connection delay before the IPv4 fallback succeeded. Replacing localhost with the explicit IPv4 loopback address 127.0.0.1 in the service configuration

removed the delay. Table II reports the effect: indexing throughput at 500 files rose from 12.4 to 19.4 files per second.

The second defect was the quadratic knowledge-graph computation described in Section IV-B. Before vectorization, rebuilding the graph at 2,000 files took 246.1 s; the same computation at 5,000 files was not measured because it was projected to take hours. After vectorization, the 2,000-file rebuild takes 24.8 s, and the 5,000-file rebuild, now measurable, takes 143.3 s. We report this last figure as the system’s current scaling limit: the reference-detection and clustering steps remain pairwise and would need an approximate method for substantially larger collections.

TABLE II
Effect of the Two Fixes on the Measured Pipeline

Measure	Before	After	Change
Index, 500 files (files/s)	12.4	19.4	1.6x
Graph rebuild, 2,000 files (s)	246.1	24.8	9.9x
Graph rebuild, 5,000 files (s)	not measured (hours, projected)	143.3	now measurable

C. Discussion

Precision at 5 reaches 1.00 at every collection size, which shows that the twelve synthetic topics separate cleanly in embedding space; it does not demonstrate retrieval quality on real personal documents, which were deliberately excluded from this evaluation. Search latency grows with collection size (50.7 ms median at 500 files to 203.5 ms at 5,000 files), tracking the growth of the vector collection queried per request. We regard the 5,000-file graph-rebuild time as the paper's most important negative result: it is a genuine, measured scaling limit of the present design, not a projection, and we report it rather than omit it.

VI. LIMITATIONS AND FUTURE WORK

Four limitations bound the claims of this paper. First, the scale evaluation uses synthetic documents; retrieval quality on a labelled set of real documents has not been measured. Second, the knowledge-graph computation remains pairwise in its reference-detection and clustering steps, so very large collections would need an approximate nearest-neighbour method. Third, the privacy audit checks configuration, not network traffic, and relies on an external tool for the independent check described in Section IV-D. Fourth, intent classification uses a 3-billion-parameter local model, which misclassifies some borderline phrasing; measured on held-out phrasings not used to tune the system, it is correct 88% of the time. Future work includes a labelled evaluation on real documents collected with consent, an approximate nearest-neighbour index for collections beyond 10,000 files, and testing whether task-specific embedding prefixes improve short-query retrieval now that a labelled evaluation set exists to judge the change.

VII. CONCLUSION

We presented DreamOS, a locally run semantic file management system that exposes embedding-based retrieval and action through a single conversational interface for a human user, with a knowledge graph, a context memory engine with deterministic precedence over model judgement, and a live, checkable privacy audit. Evaluated at up to 5,000 files with the real local models, the system surfaced and fixed two performance defects and one behavioural defect, with each fix backed by a test that fails against the prior code. We reported the resulting measurements, including the scaling limit that remains, rather than only the improvements.

ACKNOWLEDGMENT

The authors thank their project guide, Dr. Minakshi R. Rajput, for direction on this work, and the Institute of Information and Communication Technology, MGM University, for the Project Monitoring structure under which it was reviewed.

REFERENCES

[1] Z. Shi, K. Mei, M. Jin, Y. Su, C. Zuo, W. Hua, W. Xu, Y. Ren, Z. Liu, M. Du, D. Deng, and Y. Zhang, "From Commands to Prompts: LLM-Based Semantic File System for AIOS," in

Proc. Int. Conf. on Learning Representations (ICLR), 2025, arXiv:2410.11843.

[2] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient Estimation of Word Representations in Vector Space," arXiv:1301.3781, 2013.

[3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks," in Proc. Conf. on Empirical Methods in Natural Language Processing and Int. Joint Conf. on Natural Language Processing (EMNLP-IJCNLP), 2019.

[4] Chroma, "Chroma: The Open-Source Embedding Database," github.com/chroma-core/chroma, accessed 2026.

[5] Microsoft Corp., "Windows Search Overview," Microsoft Learn documentation, accessed 2026.

[6] Apple Inc., "Spotlight User Guide," support.apple.com, accessed 2026.

[7] voidtools, "Everything: Locate Files and Folders by Name Instantly," voidtools.com, accessed 2026.

[8] Ollama, "Ollama: Run Large Language Models Locally," ollama.com, accessed 2026.